

From Skill File to Knowledge Service

Exposing AMANA's proposal knowledge base to Claude through a purpose-built MCP server over Supabase.

Audience

AMANA engineering and proposal team

Decision requested

Approve Option B and Phase 1 build

The recommendation, up front

01 **Build our own MCP server. Do not point Claude at Supabase's.**

Supabase's official MCP server is a database administration interface. It hands Claude an `execute_sql` tool against our schema. Supabase's own documentation tells you not to connect it to production. We expose named retrieval tools instead.

02 **Split the skill, do not delete it.**

Roughly two thirds of amana-proposal-writer is static corpus that belongs in the DB. The remaining third is procedure (BAB7 structure, QC thresholds, expert-selection rules) that is small, stable, and cheaper to keep in the file.

03 **Hybrid retrieval, read-only, six tools maximum.**

pgvector plus Postgres full-text with metadata filters, exposed through Postgres RPC functions. No write path, no raw SQL, capped response sizes.

What the current skill actually costs us

36 KB

loaded into context on every single invocation, whether or not any of it is relevant

64%

of the file is static corpus: 10 projects, 90 consultants, 220 partners, 8 knowledge items

~60

partner names actually present. The other 160 are compressed into the phrase "and others"

Three failures follow from this

- **Lossy** The roster and partner directory are summarised to fit. Claude cannot cite an expert whose CV never made it into the file.
- **Stale** Updating a project record means editing markdown and redeploying the skill. There is no path from the DB to the skill today.
- **Blind** Claude cannot filter. It cannot ask for health-sector projects above IDR 5bn delivered for a multilateral since 2023, because there is nothing to ask.

The skill does two jobs. Only one should move.

Procedure

KEEP

Stays in the skill file

- BAB7 and CQ section structures
- QC thresholds (score ≥ 80 , ≥ 3 key numbers, ≥ 5 risks)
- Expert-selection principles
- Bahasa / English register rules
- Concept Note wizard flow

Small, stable, and needed on every draft. Retrieval here would add latency and buy nothing.

Corpus

MOVE

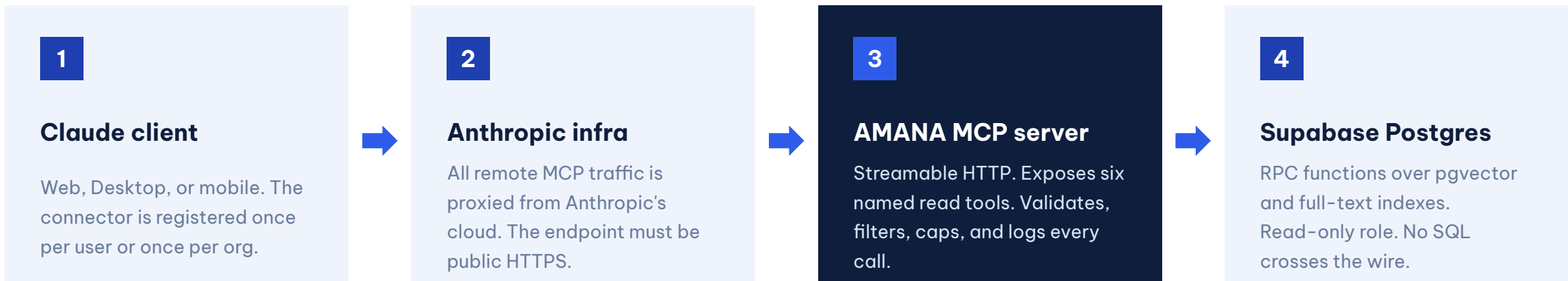
Moves to the database

- Expert roster and CVs
- Past project records and outcomes
- Partner directory
- Institutional knowledge notes
- Regulatory references

Large, changing, and needed selectively. The right mechanism is retrieval at question time.

How MCP changes the sequence

A skill is loaded before the conversation starts. An MCP tool is called during it. That is the entire difference, and it is the one that matters.



Practical consequence. Claude sees six tool descriptions at the start of a conversation, roughly 2 KB, and pulls only the records a given proposal needs. Adding the eleventh past project or the ninety-first consultant costs nothing in context and requires no redeployment.

Three ways to do this. Two of them are wrong.

A. Supabase official MCP

REJECT

- Zero build. Point Claude at `mcp.supabase.com`, authenticate, done.
- Exposes `execute_sql`. Claude writes arbitrary SQL against our schema.
- Supabase's own guidance: do not connect to production; use a dev project.
- Documented prompt-injection path (General Analysis, 2025). Read-only mode narrows it.

Right tool for schema work in Cursor. Wrong tool for a production knowledge service.

B. Custom MCP server

RECOMMEND

- Tools are named business operations. Claude cannot express a query we did not authorise.
- Server owns ranking, filtering, token caps, and provenance.
- Auth, audit, and rate limits sit at a layer we control.
- Two to four weeks of work, mostly retrieval tuning rather than protocol plumbing.

Cost is real. It is the only option that survives contact with a live proposal deadline.

C. Keep it in the skill

REJECT

- No new infrastructure, no auth, no server to operate.
- Context cost grows linearly with the corpus and is paid on every invocation.
- Every DB change requires a manual re-export and redeploy.
- Already failing today at 90 consultants and 220 partners.

This is the status quo, and the status quo is what prompted the question.

Recommended architecture

Claude clients

Custom connector registered at org level (Team/Enterprise) or per user (Pro/Max).



MCP server · Supabase Edge Function (Deno, Streamable HTTP)

Tool registry and schemas | argument validation | result ranking and token caps | untrusted-data wrapping | audit log



Postgres RPC layer · SECURITY DEFINER functions

search_corpus() · fetch_document() · match_experts() · match_projects(), run read-only



Data layer · Supabase Postgres

documents · chunks (vector + tsvector) · experts · projects · partners · org_knowledge · proposal_outcomes

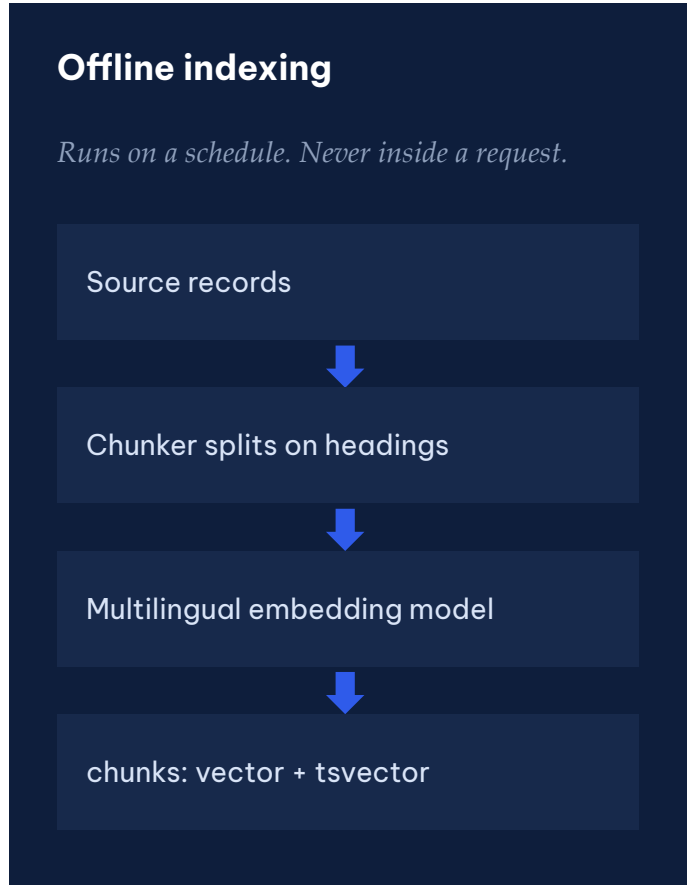
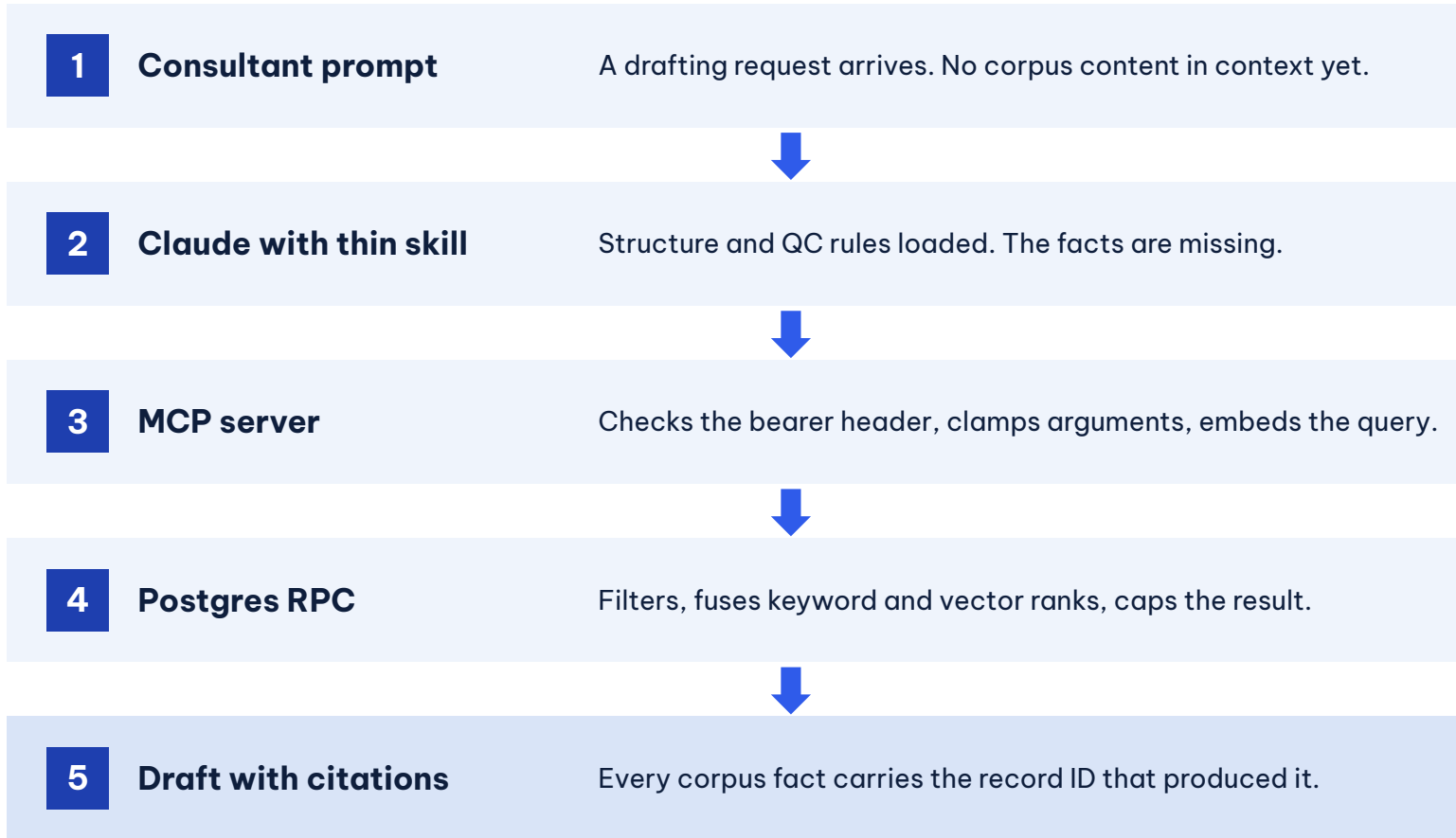
Offline pipeline

Chunking, embedding, and index refresh run as a scheduled job, not inside a tool call. A proposal draft never waits on an embedding.

Why Edge Functions

Colocated with the data, no new vendor, no separate secret store. Tradeoff: Deno runtime and cold starts. If OAuth proves painful, Cloudflare Workers ships an OAuth provider for MCP and is the fallback.

Runtime request path



What crosses each boundary

HOP	PAYLOAD	CONTROL
1 to 2	Prompt, thin skill under 10 KB, six tool schemas at roughly 2 KB	Context cost is fixed. The corpus never enters it.
2 to 3	JSON-RPC tools/call over HTTPS, proxied from Anthropic infrastructure	Arguments against a published schema. Claude never sends SQL.
3 to 4	RPC call carrying a 1024-dimension query vector and scalar filters	Executed as mcp_reader: six functions, zero table grants.
4 to 3	At most 20 rows, each snippet truncated to 700 characters	Caps live in the SQL, where no prompt can raise them.
3 to 2	Rows inside an untrusted-data boundary, plus an audit log entry	Corpus content is reference material, never instruction.
2 to 5	Draft text with a document ID against each corpus-derived claim	A claim with no ID came from the model, not the database.

What Claude actually sends at hop 2 to 3

```
{ "method": "tools/call", "params": { "name": "search_corpus", "arguments": {  
  "query": "interoperabilitas data lintas kementerian", "types": [ "project" ], "practice": "digital", "limit": 8 } } }
```

Tool surface: six tools, search then fetch

Tool descriptions are the single biggest determinant of whether Claude calls the right tool. They are prompt engineering, not documentation, and they get versioned like code.

TOOL	ARGUMENTS	RETURNS
<code>search_corpus</code>	<code>query, types[], sector, donor, year_from, limit</code>	Ranked snippets with IDs and scores. Never full documents.
<code>fetch_document</code>	<code>id, section?</code>	One full record. Called only after a search returns its ID.
<code>match_experts</code>	<code>skills[], practice, seniority, language, available_from</code>	Ranked consultants with CV extract and prior engagements.
<code>match_projects</code>	<code>sector, client_type, donor, min_value, since</code>	Comparable past projects with outcome and contract value.
<code>find_partners</code>	<code>cluster, sector, association_type</code>	Partner records for consortium and letter-of-support planning.
<code>get_template</code>	<code>structure: bab7 cq brief</code>	Section skeleton plus the QC checklist for that structure.

Non-negotiables. Every tool carries `readOnlyHint: true`. No tool accepts SQL. Every result is capped in rows and characters. Every returned record carries its source ID so drafts can be traced back to the record that produced them.

Retrieval design

1 Hybrid, not vector-only

pgvector HNSW cosine over section-level chunks, fused with Postgres full-text using reciprocal rank fusion. Semantic search alone loses exact matches on the things proposals hinge on: Permendagri 77/2020, SATUSEHAT, FHIR R4, a specific tender number.

2 Filters before ranking

Sector, donor, practice group, year, and contract value are structured columns, not embeddings. They belong in the WHERE clause. Filtering after ranking wastes the top-k budget on records that were never eligible.

3 Bilingual is a real problem

The corpus is Indonesian and English, often in the same record. Postgres ships no Indonesian stemmer. Plan on the simple configuration plus unaccent for v1, a multilingual embedding model for semantic recall, and a measured decision on a custom dictionary later.

4 Chunk on sections, return parents

Chunk at heading level so tables and roster entries stay intact. Search over chunks, return the parent record. Splitting a CV mid-table produces retrievals that are technically relevant and practically useless.

Security and authentication



Read-only by construction

The RPC functions execute as a role with SELECT only. There is no write tool, no migration tool, no SQL tool. The exfiltration leg of a prompt-injection chain does not exist.



Auth depends on our plan tier

Team or Enterprise: an org admin adds the connector once with a static header credential. Pro or Max: we implement OAuth 2.1 with DCR and PKCE. This is the largest fork in the estimate.



Never a credential in the URL

The MCP authorization spec prohibits access tokens in query strings. Headers or OAuth only. The shortcut is tempting and it leaks through logs and proxies.



Treat retrieved rows as data

Corpus rows are wrapped in an untrusted-data boundary before they reach the model. Anything a partner or client wrote into a record is content, not instruction.



Log every call

Tool name, arguments, row count, and caller, retained per call. Without this we cannot answer which records shaped a submitted proposal.

Residual risk: prompt injection is unsolved. Read-only removes the write path; human review of the draft stays the control that matters.

Risks in the request path

HOP	RISK	SIGNAL TO WATCH	MITIGATION
2	Claude never calls the tool	Zero tools/call logged in a session that produced a proposal	Descriptions state when to call, not only what the tool does. The skill forbids drafting corpus facts from memory.
2	Draft cites nothing, invents records	Share of corpus claims carrying a document ID	Citation rule in the skill. Reviewer checks three IDs per draft against the database.
3	Endpoint answers without the bearer header	Unauthenticated request returns anything but 401	Assert the 401 in CI on every deploy. Rotate the shared secret quarterly.
3	Embedding API slow or unavailable	p95 tool latency and embedding error rate	Catch the failure and fall through to the full-text branch. Degraded recall beats a dead connector on a deadline.
3	Per-query embedding cost grows with use	Monthly spend against logged call volume	Cache query embeddings keyed on a hash of the normalised query string.

Risks in the data and the organisation

HOP	RISK	SIGNAL TO WATCH	MITIGATION
4	Cross-language recall failure	recall@8 on the gold set, split by query language	Multilingual embeddings. Trigram fallback on titles. Re-run the eval on any retrieval change.
4	One call crowds the draft out of context	p95 response size in bytes per tool call	Row and character caps live in the SQL, where no prompt can raise them.
4	Privilege creep on the reader role	Schema diff showing new grants on the public schema	Assert the grant set in CI. The six RPC functions stay the only entry point.
DB	Records are summaries, so retrieval is thin	Median content length and age of updated_at	Phase 0 audit before any code is written. Treat this as a data workstream with a named owner.
ORG	Skill file and database drift apart	Any commit touching the corpus sections after cutover	Delete those sections outright. CI fails the build if they reappear.

The one control that cannot be engineered: prompt injection through corpus content. Read-only mode removes the write path and the untrusted-data wrapper reduces compliance, but neither guarantees the model ignores text addressed to it. Human review of the submitted proposal stays the control that matters.

Build sequence

Phase 0

1 week

Audit and baseline

- Confirm pgvector, schema, and record completeness
- Write 30 real proposal questions with known correct answers
- Confirm plan tier and therefore the auth model

Exit: gold question set agreed and signed off

Phase 1

2 weeks

Thin vertical slice

- Edge Function MCP server, Streamable HTTP, staging project
- Two tools only: search_corpus and fetch_document
- Full-text ranking first, no embeddings yet

Exit: 20 of 30 gold questions answered from the DB

Phase 2

2 weeks

Full surface and hardening

- Embeddings, hybrid fusion, remaining four tools
- Auth, rate limits, audit logging, response caps
- Register as an org connector

Exit: 27 of 30, with citations traceable to record IDs

Phase 3

1 week

Retire the corpus

- Rewrite the skill down to procedure only
- Delete the roster, projects, and partner sections
- Measure context reduction and draft quality against the old skill

Exit: skill under 10 KB, no quality regression

Decisions I need from you

- | | | |
|---|---------------------------------|--|
| 1 | Plan tier | Are we on Team or Enterprise? If yes, static header auth and Phase 2 loses about a week. If Pro, we build OAuth. |
| 2 | Ownership | Who operates the server after launch? An MCP endpoint is production infrastructure, not a script. |
| 3 | Source of truth | Once the corpus is in the DB, the skill file stops being authoritative. Confirm nobody keeps editing it in parallel. |
| 4 | Embedding model and cost | Multilingual capability matters more than benchmark rank here. Needs a cost line before Phase 2. |
| 5 | Corpus completeness | Does the DB hold full CVs and full project records, or the same compressed summaries the skill has? |

The fifth question is the one that decides whether this is a six-week project or a three-month one.

The argument in one sentence

Stop shipping the knowledge base and start querying it.

The skill was never the wrong idea. It was the wrong container for 220 partners and 90 consultants. Keep the procedure, move the corpus, and let Claude ask.

Next step: approve Phase 0 and name the owner.